

Convolutional neural networks per il riconoscimento di immagini meteorologiche

Candidato:

Federico Conti

Sommario

INTRODUZIONE.....	3
1.1 Apprendimento supervisionato:	3
1.2 Apprendimento non supervisionato.....	3
1. 3 Le reti neurali.....	4
1.3.1 Scelta di non linearità:.....	5
1.3.2 Inizializzazione dei pesi:	7
2. convolutional neural network	9
2.1 struttura di una cnn	9
2.2 Strati di convoluzione.....	9
2.4 strato di pooling:	11
2.5 Fully Connected	13
3. Analisi e modello.....	14
3.1 set di dati	14
3.2 Applicazione della CNN	15
3.4 transfer learning:	24
CONCLUSIONI:	30
Bibliografia e sitografia :	31

INTRODUZIONE

Al giorno d'oggi la mole di immagini che abbiamo a disposizione è molto ampia. Nei seguenti capitoli si utilizzeranno dataset contenenti immagini meteorologiche per rendere possibile la predizione futura. Si è reso basilare implementare sistemi che siano in grado di determinare il tempo meteorologico futuro. Tale sfida si traduce in un problema di classificazione a 4 stati (in base agli stati meteorologici considerati) utilizzando strumenti di deep learning, in particolare con l'applicazione delle Convolutional Neural Networks. Si è scelto di utilizzare questo procedimento, da una parte per i limiti imposti dai metodi tradizionali offerti dalla statistica , ma anche dai problemi che questi implicano dal punto di vista computazionale.

I problemi di apprendimento vengono suddivisi in due tipologie principali apprendimento supervisionato e apprendimento non supervisionato.

1.1 Apprendimento supervisionato:

L'apprendimento supervisionato ha l'obiettivo di prevedere il valore di un diverso parametro di output dato un elemento cui dici conosco i parametri (features). per fare ciò serve un modello. Quest'ultimo si definisce a partire da un insieme di X di feature considerate come l'input nel problema: ad ogni elemento viene associata una Y di output (target). L'obiettivo dell'apprendimento supervisionato è quello di prevedere il relativo output derivato da un modello che spieghi la relazione tra features e target, per consentire ad un nuovo elemento X , di predire il corrispondente valore di output Y .

1.2 Apprendimento non supervisionato

Nell' 'apprendimento non supervisionato i dati consistono soltanto un insieme edificio di X_i senza una variabile di output y . L'obiettivo è quello di individuare un modello che si adatti al meglio ai dati per scoprire interconnessioni nascoste tra di essi punto in un problema di apprendimento non supervisionato visto che cioè l'assenza di un valore corretto di output che venga preso come riferimento non è possibile misurare la correttezza della soluzione trovata rispetto una soluzione corretta quello che si può fare è quello di misurare la verosimiglianza dell'insieme degli elementi considerati rispetto al modello ottenuto. Si può realizzare una valutazione del modello, sulla base della likelihood media di tutti gli n elementi dell'insieme:

$$L(\frac{M}{T}) = \frac{1}{N} \sum_{i=1}^n P(x_i / M, T)$$

che misura la verosimiglianza dell'insieme di elementi considerati, rispetto al modello ottenuto.

1. 3 Le reti neurali

Prima di focalizzarci nella struttura delle *Convolutional Neural Networks* è necessario soffermarsi sulla natura delle reti neurali classiche. Inventate intorno alla metà degli anni 80, sono diventate dal 2010 un modello in voga grazie al miglioramento delle risorse computazionali, decretando l'inizio del deep learning, proponendo la risoluzione di nuove problematiche di analisi come la classificazione di immagini e video o il riconoscimento vocale e testuale.

Le reti neurali sono un sistema artificiale di elaborazione dell'informazione che si pone l'obiettivo di emulare il sistema nervoso animale. Quest'ultimo ha delle caratteristiche attraenti dal punto di vista un sistema di calcolo:

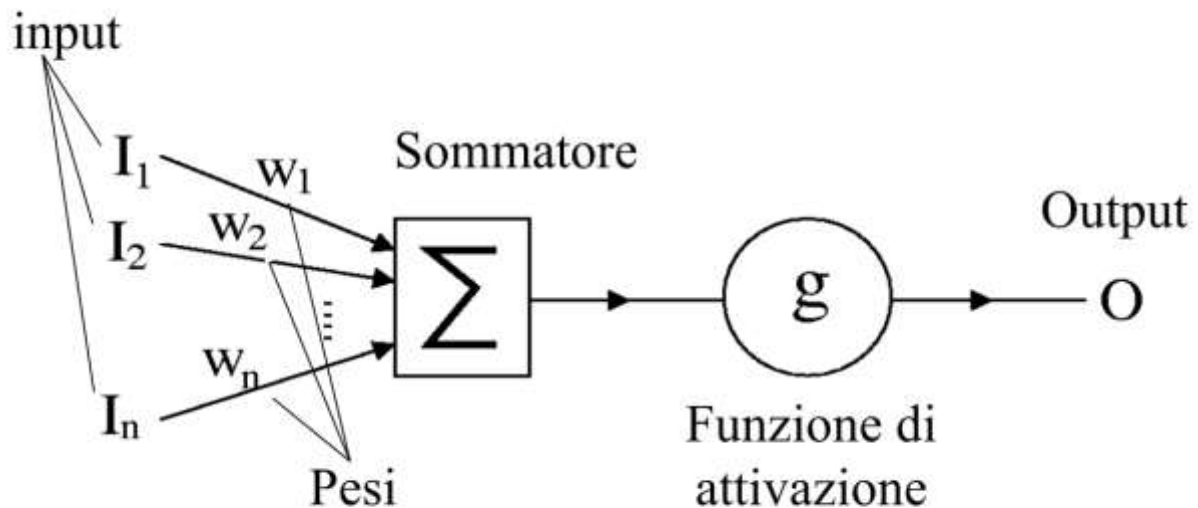
- flessibile: si adatta alle nuove situazioni;
- Permette una computazione altamente parallela;

Ha una struttura Neurale complessa composta da una parte mentale di calcolo cioè i neuroni che sono collegati tra loro tramite connessioni pesate dette sinapsi, infine, ci sono dei neuroni che sono connessi all'ambiente esterno i quali sono l'input

Ogni neurone possiede:

- Un insieme di ingressi provenienti dagli altri neuroni;
- Un'uscita verso gli altri neuroni;
- Un livello di attivazione neurone acceso o spento a cavo un sistema per calcolare il livello di attivazione al tempo.

Le componenti di un neurone sono i seguenti:



- $in_1, in_2, \dots, in_d$ sono i d ingressi che il neurone i riceve da assoni di neuroni afferenti
- $w_{1i}, w_{2i}, \dots, w_{di}$ sono i pesi che determinano l'efficacia delle connessioni sinaptiche dei dendriti
- w_{0i} è il bias, cioè un ulteriore peso che si considera collegato a un input fittizio con valore sempre a 1; il bias è utile per ricavare il punto ottimale del neurone.
- net_i è il livello di eccitazione globale del neurone
- $f(.)$ è la funzione di attivazione che determina il comportamento del neurone (ovvero il suo output) in funzione del suo livello di eccitazione net_i .

1.3.1 Scelta di non linearità:

Lo scopo principale della funzione di attivazione è introdurre nel modello la proprietà della non linearità. Nelle reti neurali artificiali, la funzione di attivazione di un nodo definisce l'output in quel nodo dato un input o un insieme di input.

La funzione di attivazione più utilizzata è la funzione sigmoide ma nel tempo è stata accantonata per problematiche a livello pratico.

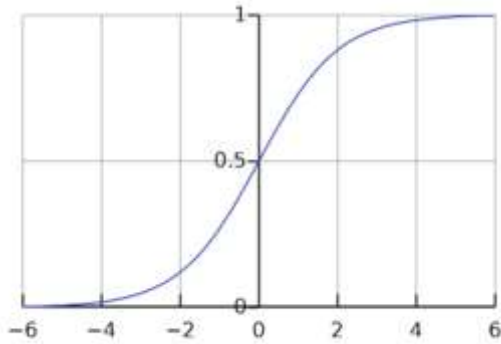
La funzione sigmoide è continua e derivabile. Per questo viene usata nei modelli di rete neurale nei quali l'algoritmo di apprendimento richiede l'intervento di formule in cui compaiano derivate.

$$f(net) = \sigma(net) = \frac{1}{1 + e^{-net}}$$

La

derivata:

$$\sigma'(net) = \frac{e^{-net}}{(1 + e^{-net})^2} = \sigma(net)(1 - \sigma(net))$$



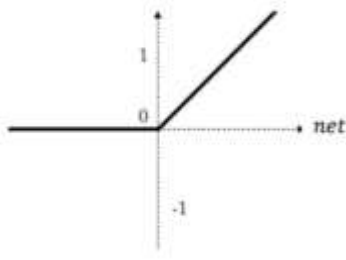
In pratica, si ha un flusso utile del gradiente solo per valori di input che rimangono all'interno di una zona di sicurezza, cioè nei dintorni dello zero. Il secondo problema deriva invece dal fatto che gli output della funzione sigmoide non sono centrati intorno allo zero, e di conseguenza gli strati processati successivamente, riceveranno anch'essi valori con una distribuzione non centrata sullo zero. Questo influisce in maniera significativa sulla discesa del gradiente, in quanto gli input in ingresso ai neuroni saranno sempre positivi, e pertanto il gradiente dei pesi associati diventerà durante la fase di back propagation, sempre positivo o sempre negativo. Tale risultato si traduce in una dinamica a zig-zag negli aggiornamenti dei pesi che rallenta in maniera significativa il processo di convergenza.

Un neurone con una funzione di attivazione ReLU accetta qualsiasi valore reale come suo input, ma si attiva solo quando questi input sono maggiori di 0.

Nel deep learning, le funzioni di attivazione sono un insieme di regole che determinano se un neurone si attiva o meno, dato una serie di input. Le funzioni che verranno spiegate rispettivamente sono la RELU e softmax.

Un neurone con una funzione di attivazione ReLU accetta qualsiasi valore reale come input, ma si attiva solo quando questi input sono maggiori di 0. Offre un sostanziale incremento

prestazionale rispetto la funzione sigmoide, per questo ad oggi è una delle più utilizzate. Poiché risolve il problema della dissolvenza del gradiente, non andando a saturare i neuroni.



$$ReLU(z) = \max(0, z)$$

$$\frac{ReLU(z)}{\partial z} = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$$

Problemi nell'utilizzo della funzione, sorgono quando il gradiente accumulato ha valori negativi, in quanto viene azzerato e i pesi non verranno aggiornati.

La funzione softmax è una generalizzazione della funzione logistica che comprime un vettore K-dimensionale Z di valori reali arbitrari in un vettore K. Mappa gli input non normalizzati in un insieme di probabilità esponenziale e normalizzate. Nel nostro contesto di studio, la funzione softmax verrà utilizzata per il problema di classificazione multi-classe per generalizzare la regressione logistica quando ci sono più di due classi di risultati.

Quindi la funzione di attivazione softmax trasforma i logit in probabilità. I valori logit sono illimitati, visto che vanno da meno infinito negativo a più infinito positivo. I Log aumentano in modo esponenziale e la velocità di aumento è molto più lenta per softmax.

La funzione softmax esegue per ogni neurone K una sorta di normalizzazione i cui valori di uscita Z_k possono essere interpretati come una probabilità:

$$z_k = f(v_k) = \frac{e^{v_k}}{\sum_{i=1}^S e^{v_i}}$$

1.3.2 Inizializzazione dei pesi:

Prima di addestrare una rete da zero, è necessario ovviamente inizializzare i pesi si noti che non si sa a priori quali debba essere il valore finale dei pesi di una rete addestrata, ma è ragionevole assumere che approssimativamente la metà dei paesi abbia un valore negativo e l'altra metà un valore positivo. Impostando tutti i valori a zero per i pesi tutti i neuroni di ciascun strato avrebbero lo stesso ruolo in uscita quindi anche i valori dei gradienti sarebbero

tutti uguali così come tutti ori di aggiornamento in poche parole quando si inizializzato tutti i paesi a zero non si creano asimmetrie tra i neuroni.

Ci vuole quindi che inizialmente i pesi siano prossime allo zero ma allo stesso tempo non siano pari a zero una soluzione è quella di inizializzare i pesi con un numero sufficientemente piccolo creando asimmetria tra essi quindi si vanno a creare dei neuroni distinti tra loro in modo che essi possano valutare valori diversi degli aggiornamenti ed integrarsi auto mente come parti diverse di un'unica rete. Questa soluzione può soddisfare nel caso di una rete con pochi strati nascondi, altrimenti durante la fase back propagation il gradiente accumulato continua ad essere moltiplicato per quantità vicine allo zero che porteranno alla sua dissolvenza.

2. convolutional neural network

2.1 struttura di una cnn

Le *convolutional neural network* sono reti neurali specializzate nell'elaborazione di dati che presentano una struttura a griglia, come ad esempio immagini sia in bianco e nero o a colori e anche le serie storiche, che possono essere pensate come una griglia monodimensionale di campionamento.

La rete prende questo nome perché utilizza un'operazione matematica lineare chiamata convoluzione, di conseguenza una rete neurale classica che implementa operazioni di convoluzione in almeno uno dei suoi strati, viene definita Convolutional.

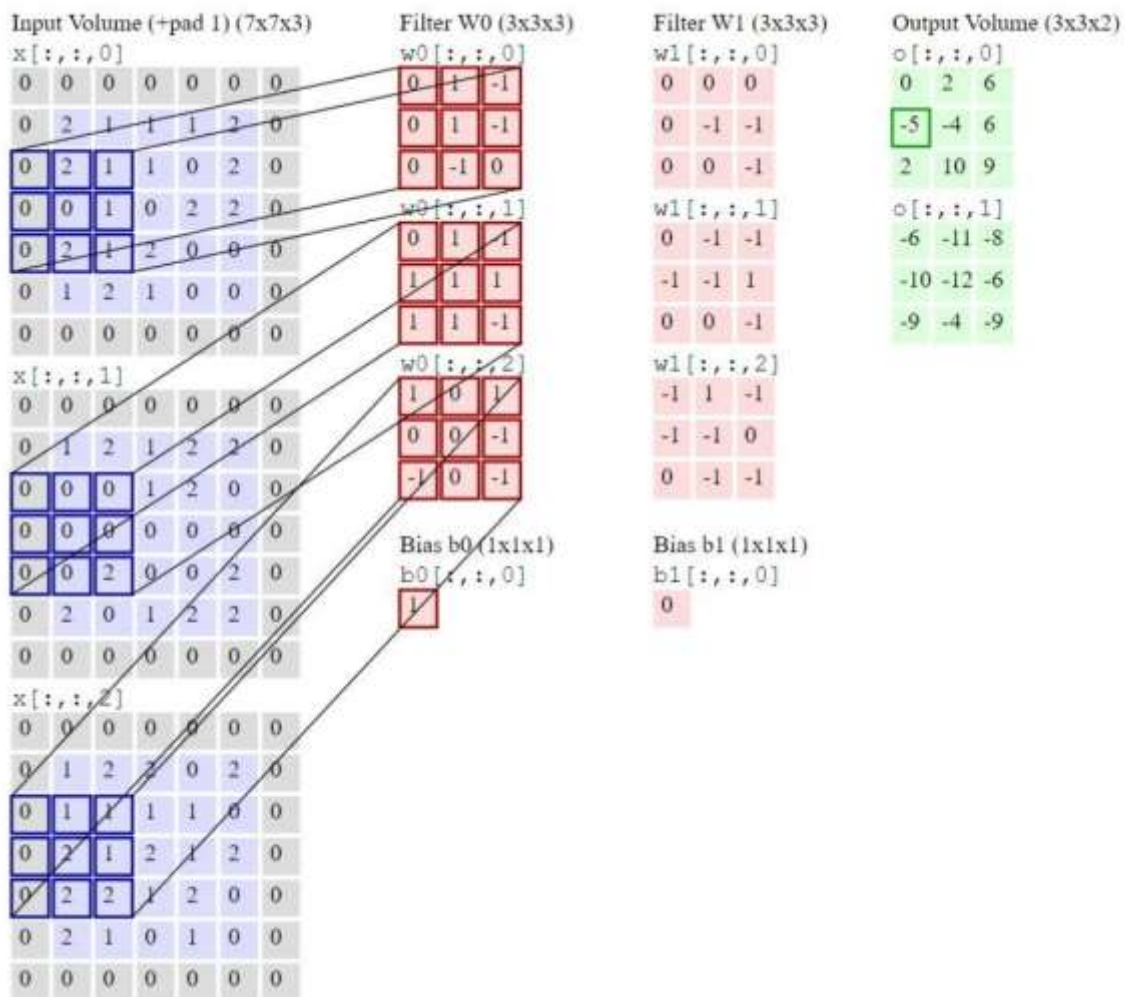
Gli strati composti da operazioni di convoluzione prendono il nome di convolutional layers, ma non sono gli unici strati che compongono una CNN: nella struttura tipicamente si ha l'alternarsi di Convolutional layers, pooling layers e fully connected layers.

2.2 Strati di convoluzione

Gli strati di convoluzione sono composti da neuroni organizzati in 3 dimensioni: larghezza, altezza e profondità, dove per profondità si intende la profondità dell'immagine, non alla profondità della rete, che invece corrisponde al numero di strati.

L'elaborazione locale e la condivisione dei pesi rendono il modello più semplice ed efficace. L'elaborazione locale fa sì che i neuroni processino nella stesso modo diverse parti di un'immagine. Alla base di uno strato di convoluzionale c'è l'operazione di convoluzione, cioè l'applicazione di filtri digitali. Un filtro bidimensionale viene fatto scorrere nell'immagine eseguendo il prodotto scalare tra la porzione dell'input coperta e il filtro.

La convolutional layers è formato da un set di filtri molto numeroso, Nf tutti con la medesima estensione spaziale. Ogni filtro viene fatto convolvere lungo la larghezza e l'altezza del volume di input, e pertanto vengono prodotte Nf mappe di attivazioni bidimensionali, le quali forniscono ognuna la risposta del relativo filtro in ogni posizione spaziale. La loro concatenazione lungo la terza dimensione produce l'output del Convolutional Layer.



Nell'immagine sovrastante, si nota l'operazione convoluzione di due differenti filtri (W_0 e W_1) di dimensione 3 X 3 X 3 su un volume di input 7 X 7 X 3.

É facile notare che gli indici dei pesi che definiscono il singolo neurone non dipendono da (i, j), ovvero dalla sua posizione nella mappa di attivazione, ma solamente dal filtro generatore f. In altre parole, questo significa che i neuroni appartenenti alla stessa mappa di attivazione condividono sempre lo stesso set di pesi, ossia quelli del filtro che li ha generati. Questo definisce la seconda proprietà fondamentale delle Convolutional Neural Networks, ossia il weight sharing.

Disposizione finale e numero di neuroni che compongono il volume di output del Convolutional layer sono controllati da 3 iperparametri: profondità. Stride e zero-padding.

- Profondità: definita come il numero di filtri N_g che compongono lo strato, ognuno con caratteristiche differenti nel volume di input.
- Stride: il numero di pixel di cui si vuole traslare il filtro ad ogni spostamento. Quando lo stride è pari ad uno significa che stiamo muovendo il filtro un pixel alla volta, e di conseguenza viene scannerizzata ogni possibile posizione dell'input. Valori più alti muovono il filtro con salti maggiori, e pertanto viene generato un output di dimensioni minori.
- Zero- padding: per preservare il numero di neuroni in orizzontale e in verticale, si aggiunge un padding, cioè un bordo intorno all'immagine iniziale, posto pari a 0 o posto pari al neurone più vicino. Ad esempio, per filtri 5 X 5 si aggiungerà un padding pari a 2.

Nelle immagini reali, l'input non è bidimensionale ma tridimensionale, definendo i colori dell'immagine. La terza dimensione del filtro eguaglia la terza dimensione dell'input e la convoluzione avviene in modo analogo.

Il risultato di una convoluzione tridimensionale è bidimensionale, ma non viene utilizzato un solo filtro ma una serie di filtri e il risultato ottenuto dai vari filtri viene posto in un tensore tridimensionale di output la cui profondità è pari al numero di filtri. Ogni matrice che si ottiene dall'applicazione di un filtro viene chiamata feature map.

2.4 strato di pooling:

Lo strato di pooling serve per aggregare le caratteristiche del livello precedente generando delle feature maps di dimensioni più piccole attraverso un downsampling. Lo scopo di scendere di dimensionalità è quello di conferire invarianza alle trasformazioni dell'input mantenendo le informazioni significati a discriminare.

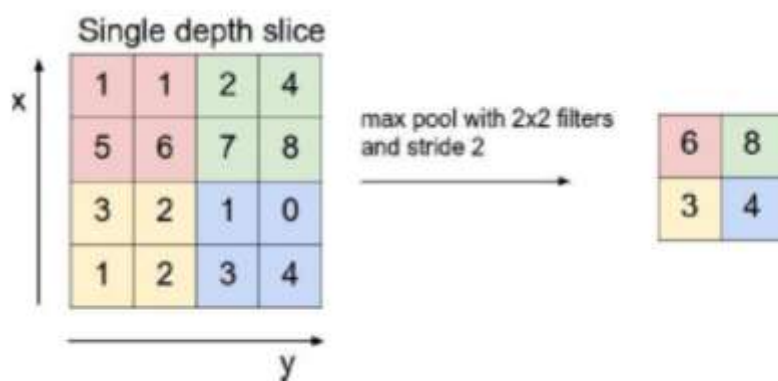
Questo serve perché il livello di convoluzione introduce due problemi:

- Complessità
- Overfitting

Le reti convoluzionali possono includere livelli di pooling locali o globali per semplificare il calcolo sottostante. I livelli di pooling riducono le dimensioni dei dati combinando le uscite dei cluster di neuroni in uno strato in un singolo neurone nel successivo strato. Infatti, eseguendo uno strato di convoluzione, è comune trovare uno strato di sotto campionamento che riduce la dimensione delle matrici di output. Le operazioni più comuni sono:

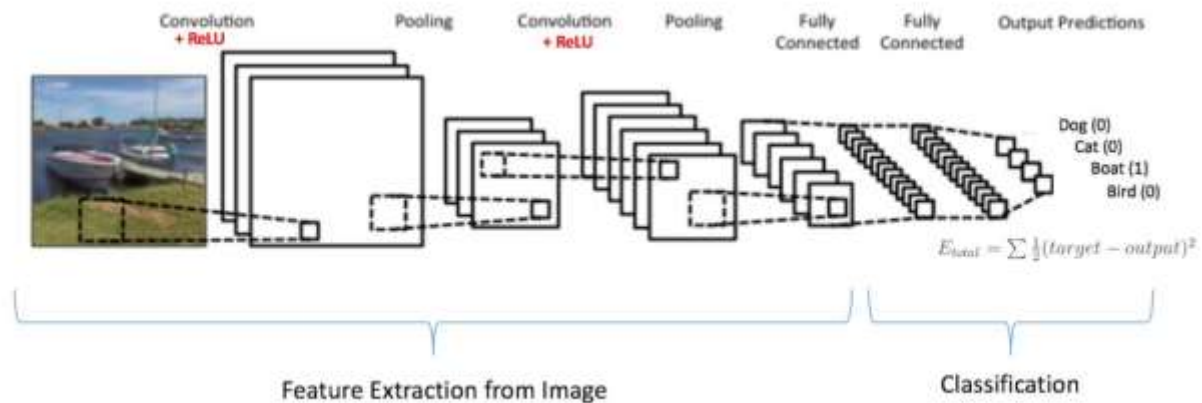
- Max Pooling: ogni cella della matrice risultante contiene il valore massimo della sottomatrice di partenza.
- Average Pooling: scende di dimensionalità prendendo la media degli input della finestra.

La forma più comune di pooling è il max-pooling 2×2 , che dimezza le due dimensioni prendendo il massimo calcolato su regioni 2×2 , spostandosi di due posizioni ad ogni iterazione.



La dimensione della finestra e la quantità dello spostamento sono due iperparametri diversi. È possibile aggiungere dei padding nei bordi anche se viene fatto meno frequentemente per gli strati convoluzionali. L'avg-pooling opera in modo analogo, calcolando la media invece che il massimo dei valori

2.5 Fully Connected



La fase di fully connected è una componente della rete neurale che opera sul volume di output vettorizzato dello strato che lo precede. Nella CNN, gli strati possono essere visti come vettori di neuroni. L'output di ciascun neurone viene inviato a tutti i neuroni dello strato successivo con diversi pesi. La funzione principale degli strati, inseriti solo all'ultimo per completare la struttura della rete ha lo scopo di raggruppare le informazioni ottenute negli strati precedenti esprimendole con l'attivazione neuronale che servirà nei successivi calcoli per la classificazione finale per ottenere prestazioni migliori da dettagli che non sono stato esplorati.

3. Analisi e modello

3.1 set di dati

Per sviluppare la CNN, ci sono stati consegnati due diversi set di dati, entrambi contenente immagini sulle condizioni meteorologiche. Il primo denominato MWI è organizzato in un insieme di directory, ognuna delle quali contiene altre quattro directory (una per classe) contenenti 100 immagini per classe. potremmo usare l'insieme completo del database o qualsiasi suo sottoinsieme. Il secondo è stato creato da una piccola azienda denominata SMART-I che costruisce sensori intelligenti e consiste di 3000 immagini sulle condizioni meteorologiche, vale a dire nevoso, soleggiato e piovoso e sono per la maggior parte immagini di strade.

Queste immagini hanno molto rumore e non sempre di buona qualità; quindi, ho dovuto rimodellarle per rendere le dimensioni dell'immagine coerenti con il livello di input della rete. Ho deciso di utilizzare quattro set di dati MWI (MWI 1.1.1, 1.1.2, 1.1.3, 1.1.4 e 1.1.5) per addestrare il modello e un set di dati MWI per testare l'accuratezza del modello (MWI 1.2.3) e il blind test fornitoci per fare previsioni.

Haze:



Rainy:



Snowy:



Sunny:



3.2 Applicazione della CNN

```
tensorflow_version 1.x

import numpy as np
import tensorflow as tf
import keras
from keras.preprocessing.image import ImageDataGenerator
from keras.applications.imagenet_utils import preprocess_input, decode_predictions
from keras.layers import Input, Add, Dense, Activation, ZeroPadding2D, BatchNormalization, Flatten, Conv2D, AveragePooling2D, MaxPooling2D, GlobalMaxPooling2D
from keras.models import Model
from keras.models import load_model
from keras.preprocessing import image
from keras.applications.resnet50 import ResNet50
from keras.utils import layer_utils
from keras.utils.data_utils import get_file
from keras.applications.imagenet_utils import preprocess_input
from keras.utils.vis_utils import model_to_dot
from keras.utils import plot_model
from keras.initializers import glorot_uniform
import matplotlib.pyplot as plt

def plot_confusion_matrix(y_true, y_pred, classes,
                          normalize=False,
                          title=None,
                          cmap=plt.cm.Blues):
```

Per realizzare il progetto ho utilizzato la libreria keras, utilizzata per l'apprendimento automatico e le reti neurali scritta in python. Inoltre, ho scritto la funzione della matrice di confusione per utilizzarla successivamente.

Ho importato il set di dati da Google drive

```
from google.colab import drive
drive.mount('/content/drive')
datadir = '/content/drive/My Drive/CNN'
trainingset = '/content/drive/My Drive/CNN/Train'
testset = '/content/drive/My Drive/CNN/MWI-Dataset-1.2.3/MWI-Dataset'
```

Dopodiché ho inizializzato le variabili per definire l'altezza e la larghezza dell'immagine, il colore, il numero di batch e di epoche. La dimensione del batch è data dall'iperparametro che definisce il numero di campioni su cui lavorare prima di aggiornare i parametri interni al parametro, mentre il numero di epoche è un iperparametro che definisce il numero di volte in cui l'algoritmo di apprendimento lavorerà sull'intero set di dati di training

Load immagini:

per caricare il set di dati dalle directory appropriati, ho usato ImageDataGenerator messo a disposizione da keras. L'ho utilizzato due volte, uno per il set di training e per set di convalida finale. Nel primo caso ho utilizzato:

- Rescale
- zoom_range
- rotation_range
- width_shift_range
- height_shift_range
- horizontal_flip
- vertical flip

```

] train_datagen = ImageDataGenerator(
    rescale = 1. / 255,\
    zoom_range=0.1,\
    rotation_range=10,\
    width_shift_range=0.1,\
    height_shift_range=0.1,\
    horizontal_flip=True,\
    vertical_flip=False)

train_generator = train_datagen.flow_from_directory(
    directory=trainingset,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    color_mode=color_mode,
    batch_size=batch_size,
    class_mode="categorical",
    classes=['HAZE', 'RAINY', 'SNOWY', 'SUNNY'],
    shuffle=True
)

test_datagen = ImageDataGenerator(
    rescale = 1. / 255)

test_generator = test_datagen.flow_from_directory(
    directory=testset,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    color_mode=color_mode,
    batch_size=batch_size,
    class_mode="categorical",
    classes=['HAZE', 'RAINY', 'SNOWY', 'SUNNY'],
    shuffle=False
)

```

Queste caratteristiche grazie alla libreria keras hanno reso possibile l'aumento dei dati per espandere artificialmente le dimensioni di un set di dati di training creando versioni modificate delle immagini nel set di dati.

L'addestramento di modelli di rete neurale di deep learning su più dati può portare a modelli più abili e le tecniche di aumento possono creare variazioni delle immagini che possono migliorare la capacità dei modelli di adattamento di generalizzare ciò che hanno appreso a nuove immagini. La libreria di rete neurale di deep learning di Keras offre la capacità di adattare i modelli utilizzando l'aumento dei dati di immagine tramite la classe *ImageDataGenerator*.

L'aumento dei dati delle immagini è forse il tipo più noto di aumento dei dati e implica la creazione di versioni trasformate delle immagini nel set di dati di addestramento che appartengono alla stessa classe dell'immagine originale.

Le trasformazioni includono una serie di operazioni dal campo della manipolazione delle immagini, come spostamenti, capovolgimenti, zoom e molto altro.

L'obiettivo è di espandere il set di dati di addestramento con nuovi esempi plausibili, dando delle variazioni delle immagini al set di training, come ad esempio un capovolgimento orizzontale dell'immagine.

L'aumento dei dati è stato applicato esclusivamente al set di training e non al set di validation.

```
def build_model():
    model = Sequential()
    model.add(Conv2D(filters = 32, kernel_size = (5,5),padding = 'Same', activation = 'relu', input_shape = (IMG_HEIGHT, IMG_WIDTH, dim)))
    model.add(Conv2D(filters = 32, kernel_size = (3,3),padding = 'Same', activation = 'relu'))
    model.add(MaxPool2D(pool_size=(2,2)))
    model.add(Dropout(0.25))

    model.add(Conv2D(filters = 64, kernel_size = (5,5),padding = 'Same', activation = 'relu'))
    model.add(Conv2D(filters = 64, kernel_size = (3,3),padding = 'Same', activation = 'relu'))
    model.add(MaxPool2D(pool_size=(2,2), strides=(2,2)))
    model.add(Dropout(0.25))

    model.add(Flatten())
    model.add(Dense(256, activation = "relu"))
    model.add(Dropout(0.5))
    model.add(Dense(4, activation = "softmax"))

    model.compile(
        optimizer='adam',
        loss=['categorical_crossentropy'],
        metrics=['accuracy']
    )
    return model

model = build_model()
model.summary()
```

Ho creato un modello sequenziale composto da 12 strati. Gli strati convoluzionali fanno la convoluzione tra l'immagine e il numero di kernel che vanno ad enfatizzare le caratteristiche dell'immagine come i bordi, la differenza nella colorazione, le forme dentro l'immagine e ecc. Ho utilizzato 4 strati convolutivi con differenti forme e dimensioni del kernel. Il primo e il terzo strato hanno una dimensione del kernel pari a 5,5 mentre il secondo e il quarto hanno una dimensione del kernel pari a 3,3. Ogni strato convoluzionale ha utilizzato la funzione di attivazione RELU. Inoltre, ho utilizzato due livelli MaxPool2D con una dimensione 2,2 e tre strati Dropout per ridurre l'overfitting.

Il dropout è una tecnica di regolarizzazione per ridurre l'overfitting nelle reti neurali artificiali prevedendo co-adattamenti sui dati di training. Con il dropout, la rete addestrata come la media di molte reti neurali, questo rende la rete stessa più simile a un modello di ensemble. In particolare, l'ultimo strato ha una funzione di attivazione "softmax" e un numero di output pari a 4, il numero delle classi meteorologiche.

Layer (type)	Output Shape	Param #
conv2d_17 (Conv2D)	(None, 128, 128, 32)	2432
conv2d_18 (Conv2D)	(None, 128, 128, 32)	9248
max_pooling2d_9 (MaxPooling2D)	(None, 64, 64, 32)	0
dropout_13 (Dropout)	(None, 64, 64, 32)	0
conv2d_19 (Conv2D)	(None, 64, 64, 64)	51264
conv2d_20 (Conv2D)	(None, 64, 64, 64)	36928
max_pooling2d_10 (MaxPooling2D)	(None, 32, 32, 64)	0
dropout_14 (Dropout)	(None, 32, 32, 64)	0
flatten_5 (Flatten)	(None, 65536)	0
dense_9 (Dense)	(None, 256)	16777472
dropout_15 (Dropout)	(None, 256)	0
dense_10 (Dense)	(None, 4)	1028
Total params: 16,878,372		
Trainable params: 16,878,372		
Non-trainable params: 0		

3.3 Valutazione del modello e risultati:

Una volta creato il modello, l'ho addestrato. Per farlo ho usato il file fit_generator. Per evitare il sovradattamento non ho usato solo i Dropout Layers, ma ho usato anche i early-stopping.

Ho impostato il numero di batch pari a 32, il numero di epoche pari a 53 e gli step per epoca pari a 15 sia per la validazione che per l'addestramento.

```
steps_per_epoch=train_generator.n//train_generator.batch_size
val_steps=test_generator.n//test_generator.batch_size+1
from keras import callbacks
from keras.callbacks import ModelCheckpoint
checkpoint = ModelCheckpoint("checkpoint.h5", monitor='val_acc', verbose=1, save_best_only=True, save_weights_only=False, mode='auto', period=1)
stopping = keras.callbacks.EarlyStopping(monitor='val_acc', patience=10)

try:
    history = model.fit_generator(train_generator, epochs=epochs, verbose=1,\
                                callbacks=[stopping, checkpoint],\
                                steps_per_epoch=15,\
                                validation_data=test_generator,\
                                validation_steps=15)
except KeyboardInterrupt:
    pass
```

```
[ ] import sklearn.metrics
from sklearn.metrics import classification_report, confusion_matrix

val_steps=test_generator.n//test_generator.batch_size+1

test_generator = test_datagen.flow_from_directory(
    directory=testset,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    color_mode=color_mode,
    batch_size=batch_size,
    class_mode="categorical",
    shuffle=False
)

preds = model.predict_generator(test_generator,verbose=1,steps=val_steps)

Ypred = np.argmax(preds, axis=1)
Ytest = test_generator.classes # shuffle=False in test_generator

print(classification_report(Ytest, Ypred, labels=None, target_names=classnames, digits=3))
```

```
Found 400 images belonging to 4 classes.
13/13 [=====] - 5s 368ms/step
```

	precision	recall	f1-score	support
HAZE	0.908	0.790	0.845	100
RAINY	0.692	0.450	0.545	100
SNOWY	0.587	0.810	0.681	100
SUNNY	0.827	0.910	0.867	100
accuracy			0.740	400
macro avg	0.754	0.740	0.734	400
weighted avg	0.754	0.740	0.734	400

Per valutare il modello ho utilizzato la funzione `valute_generator` che mi ha permesso di misurare l'accuracy e la precision. I risultati sono i seguenti:

La precision è il rapporto tra il numero delle previsioni corrette di una classe sul totale delle volte che il modello le prevede:

$$TF / (TF + TN)$$

Recall misura la sensibilità del modello, è il rapporto tra le previsioni corrette per una classe sul totale dei casi in cui si verifica effettivamente

$$TP / (TP + FN)$$

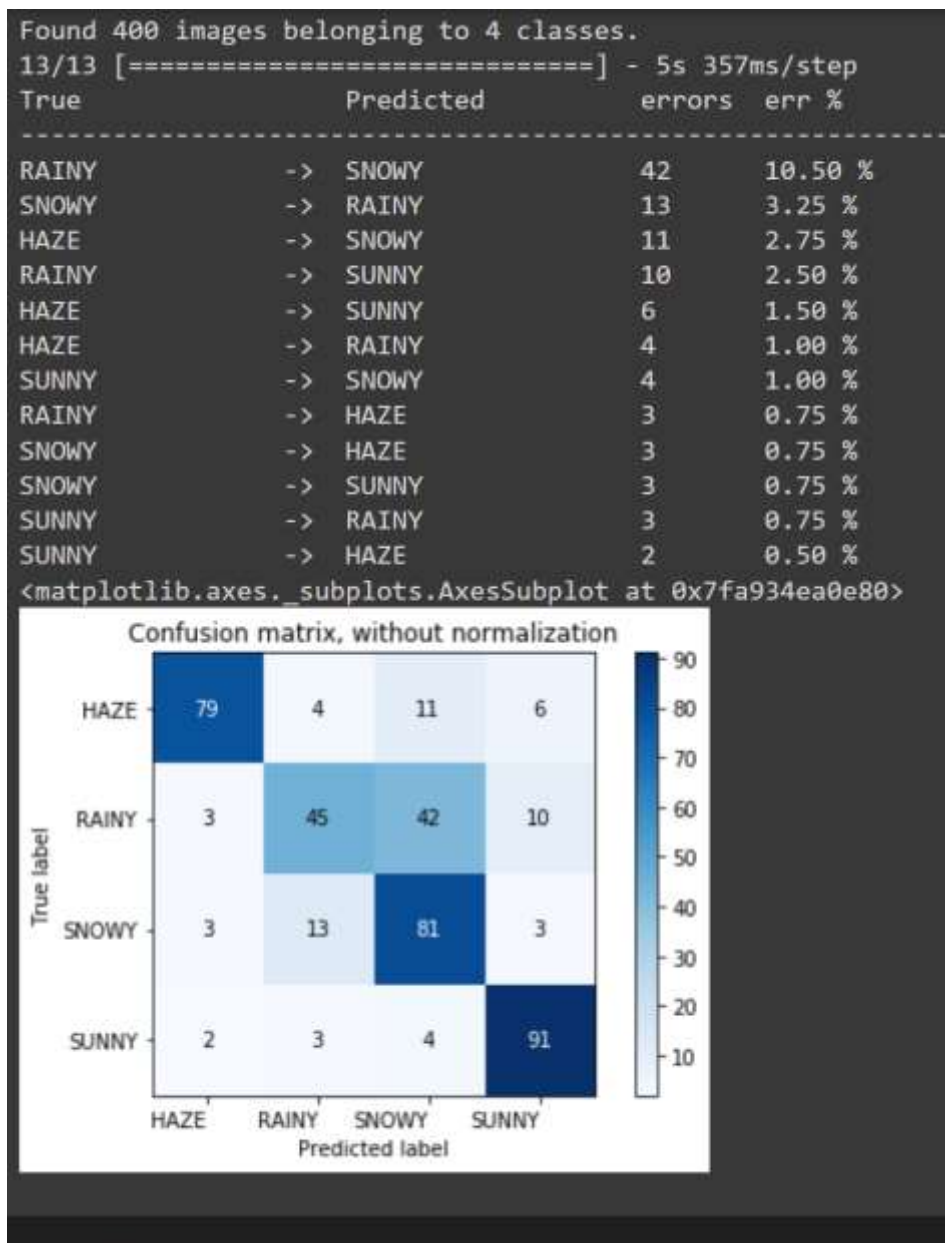
F1 score è una media armonica tra precision e recall. Si utilizza la media armonica per dare maggior peso ai valori piccolo, facendo sì che si ottenga un valore alto di F1 score solo quando precision e recall sono alti.

```
Found 400 images belonging to 4 classes.
13/13 [=====] - 5s 368ms/step
```

	precision	recall	f1-score	support
HAZE	0.908	0.790	0.845	100
RAINY	0.692	0.450	0.545	100
SNOWY	0.587	0.810	0.681	100
SUNNY	0.827	0.910	0.867	100
accuracy			0.740	400
macro avg	0.754	0.740	0.734	400
weighted avg	0.754	0.740	0.734	400

L'accuracy è la percentuale di classificazioni corrette, il 74% dei casi il modello classifica correttamente la target. Un ulteriore fattore da considerare è la precision, cioè la percentuale di positivi trovati. La precision per HAZE è 0.908. La peggiore è per la condizione SNOWY del 0,587.

La recall più alta la troviamo per SUNNY e di conseguenza si trova F1-score più alto rispetto le altre 3 classi. Una recall alta con una precision bassa, lo troviamo nella classe SNOWY, il che indica che lo prevede ogni volta che si verifica ma lo prevede anche quando non si verifica, per questo la precision assume un valore del 0.587.



La confusion matrix ha una forma tabellare che mette in relazione i valori predetti mentre ogni riga rappresenta i valori reali. E' una tabella di errata classificazione che restituisce una rappresentazione dell'accuratezza di classificazione statistica.

Il tasso di errore è la somma degli errori di classificazione diviso per il numero totale dei casi.

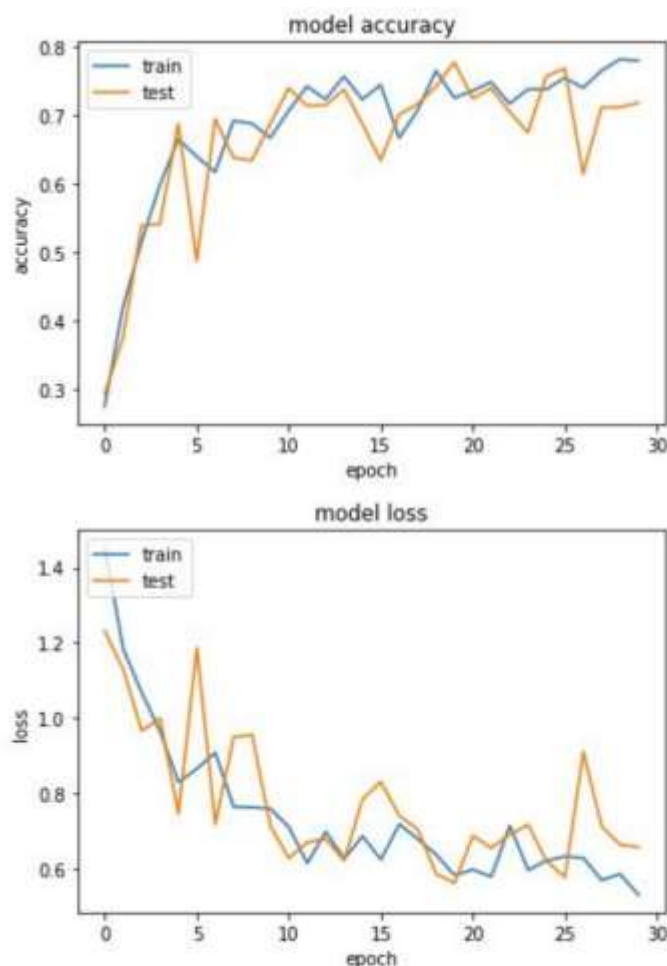
L'EER è dato dalla seguente formula:

$$ERR = FP + FNTN + FP + TP + FN$$

Il modello analizza 400 immagini. In 296 casi il modello classifica correttamente mentre per 104 casi sbaglia.

Il modello riesce a classificare meglio le immagini SUNNY, invece ha un alto tasso d'errore del 10,50% tra la classificazione delle immagini RAINY e SNOWY, il caso opposto ha il secondo tasso di errore del 3.25%. L'errore più piccolo lo troviamo nella classificazione tra immagini SUNNY e HAZE con un errore del solo 0.50%.

Infine, ho generato due grafici per analizzare l'evoluzione della precision e dell'accuracy attraverso le epoche. Da questi grafici si può dedurre che il modello non è in overfitting in quanto l'accuracy e la precision sono molto simili tra training set e validation set.



3.4 transfer learning:

Il transfer learning è un metodo di machine learning che riqualifica modelli già realizzati per rendere il prodotto finale migliore. La conoscenza pre-addestrata senza riqualificarli è l'approccio più comodo. Il transfer learning è il miglioramento dell'apprendimento automatico in un nuovo compito. Si utilizza l'output del modello precedente come input per un nuovo modello per risolvere un fine diverso ma comunque correlato con il precedente problema.

Per raggiungere il massimo beneficio nel secondo apprendimento, le caratteristiche utilizzate nel primo apprendimento devono essere sufficientemente generali in modo che il secondo modello si adatti alla nuova problematica e non ad uno specifico modello.

Lo scopo del transfer learning è quello di diminuire il tempo di addestramento del secondo modello e/o aumentare le prestazioni.

I principali vantaggi del transfer learning sono:

- Maggior performance:
- la performance durante la fase di training è maggiore
- Asintoto superiore: l'abilità convergente del modello addestrato è migliore rispetto il precedente.

```

def transferNet(feature_extractor, num_classes, output_layer_name, trainable_layers):

    # get the original input layer tensor
    input_t = feature_extractor.get_layer(index=0).input

    # set the feature extractor layers as non-trainable
    for idx, layer in enumerate(feature_extractor.layers):
        if layer.name in trainable_layers:
            layer.trainable = True
        else:
            layer.trainable = False

    # get the output tensor from a layer of the feature extractor
    output_extractor = feature_extractor.get_layer(name = output_layer_name).output

    # output_extractor = MaxPooling2D(pool_size=(4,4))(output_extractor)

    # flat the output of a Conv layer
    flatten = Flatten()(output_extractor)
    flatten_norm = BatchNormalization()(flatten)

    # add a Dense layer
    dense = Dropout(0.4)(flatten_norm)
    dense = Dense(200, activation='relu')(dense)
    dense = BatchNormalization()(dense)

    # add a Dense layer
    dense = Dropout(0.4)(dense)
    dense = Dense(100, activation='relu')(dense)
    dense = BatchNormalization()(dense)

    # add the final output layer
    dense = BatchNormalization()(dense)
    dense = Dense(num_classes, activation='softmax')(dense)

    model = Model(input=input_t, output=dense, name="transferNet")

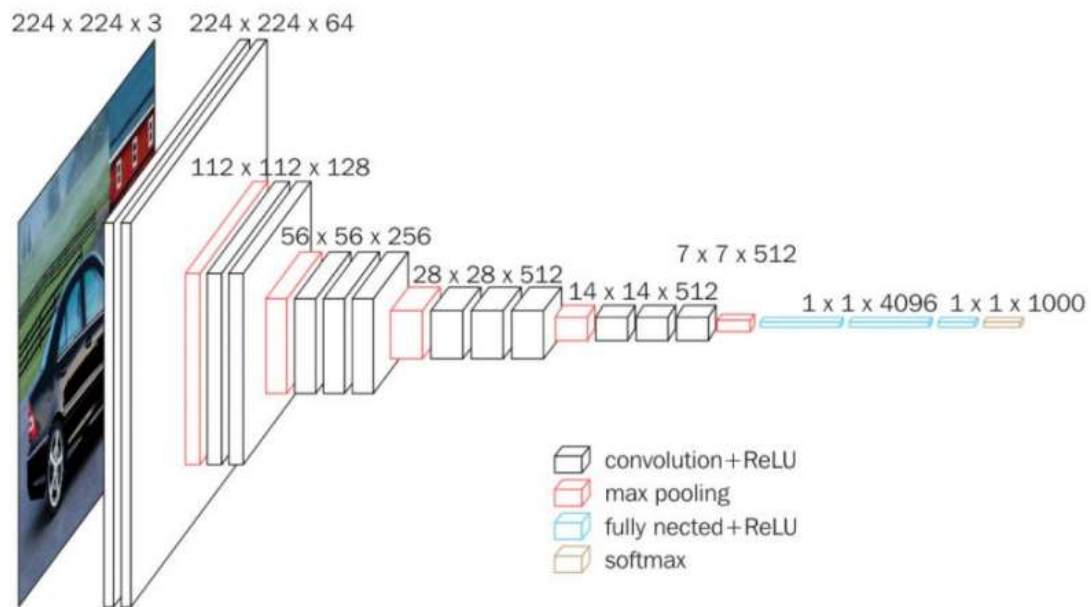
    optimizer = 'adam' #alternative 'SGD'
    model.compile(loss=keras.losses.categorical_crossentropy, optimizer=optimizer, metrics=['accuracy'])

    return model

```

è stato utilizzando in primis importando lo stesso set di dati, eseguendo l'inizializzazione e caricando le immagini. La differenza rispetto al caso sopra, è la diversa costruzione del modello. In questo caso, il modello è già pre-addestrato, in particolare VGG16, e aggiunto layers. In particolare, ho aggiunto uno strato denso con l'output con il numero di classi che sono state studiate. Inoltre, è stata utilizzando la funzione di attivazione Softmax.

VGG16 è un modello di CNN proposto da K. Simonyan e A. Zisserman, dove vengono ridotte le dimensioni del kernel con filtri di dimensioni 3x3.



Avendo l'obiettivo di migliorare l'accuracy, il modello consiste nel troncare l'ultimo strato della rete pre-addestrata e sostituirla con il nuovo strato con la funzione softmax, usando un tasso di apprendimento per addestrare la rete e in variando i pesi dei primi strati della rete performata. Anche in questo caso ho utilizzando la modalità early stopping, per non incorrere nell'overfitting.

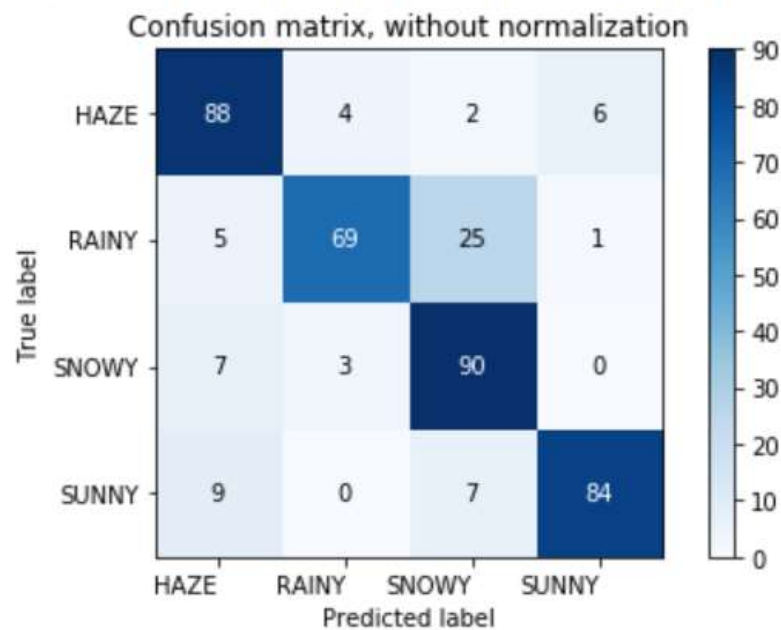
Rispetto al modello precedente, ho ottenuto una migliore precision. Infatti, la valutazione che ho ottenuto è il seguente:

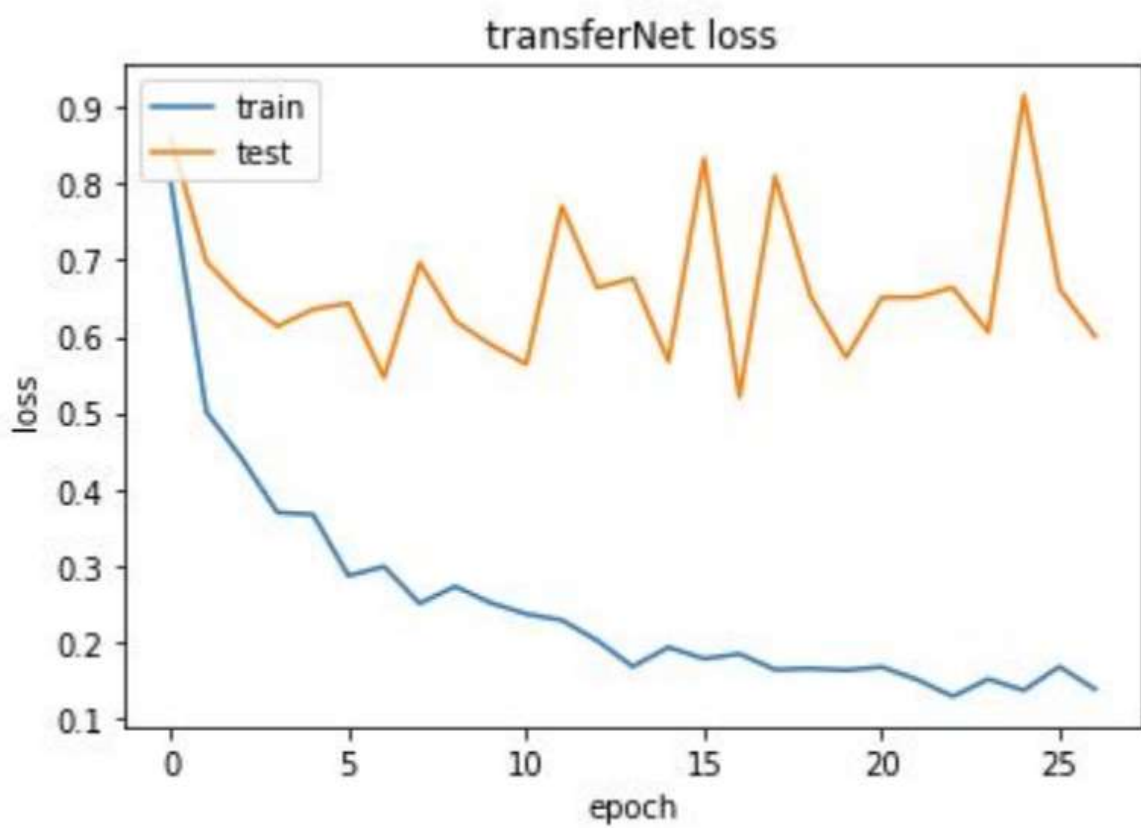
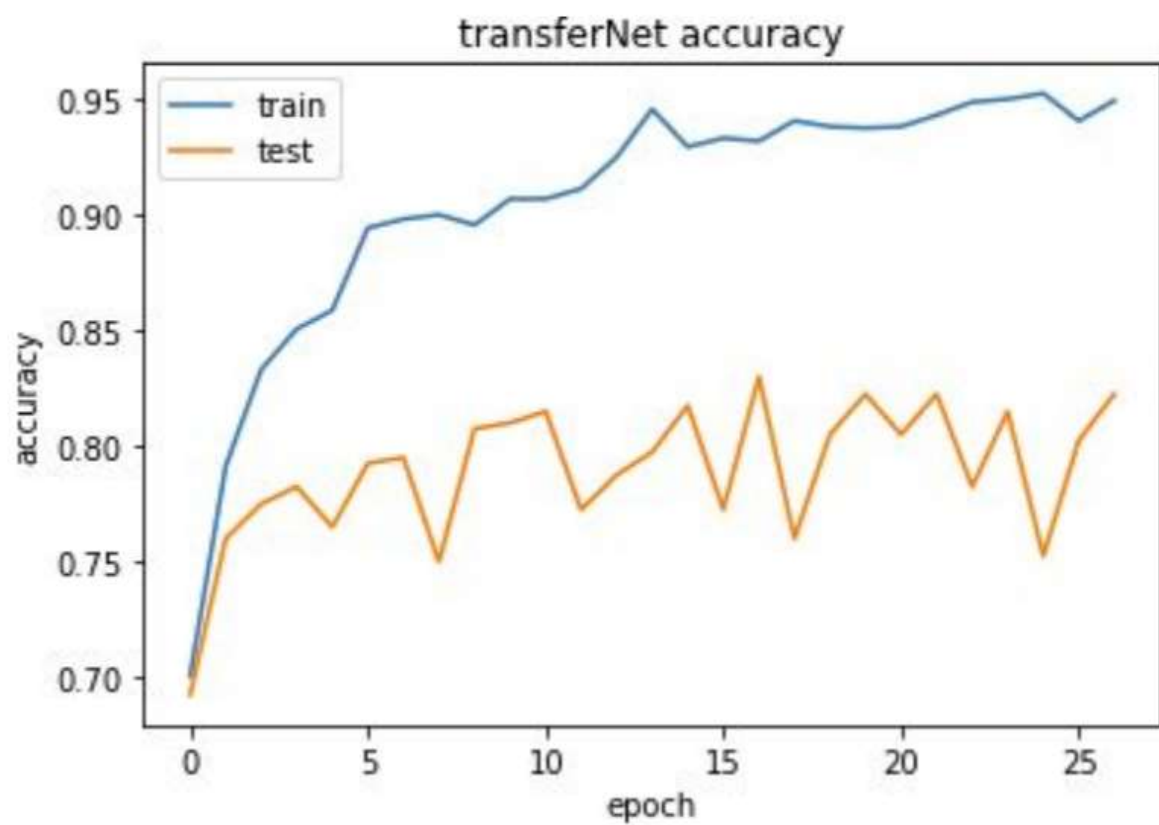
```
Found 400 images belonging to 4 classes.
13/13 [=====] - 4s 340ms/step
Test loss: 0.611227
Test accuracy: 0.805000
Found 400 images belonging to 4 classes.
13/13 [=====] - 4s 346ms/step
True           Predicted      errors  err %
```

L'accuracy assume un valore di 0.82 rispetto l'accuracy precedente dello 0,765. Il modello come il precedente prevede al meglio i casi di SUNNY e SNOWY. Gli errori rimangono invariati rispetto al caso precedente

RAINY	->	SNOWY	25	6.25 %
SUNNY	->	HAZE	9	2.25 %
SNOWY	->	HAZE	7	1.75 %
SUNNY	->	SNOWY	7	1.75 %
HAZE	->	SUNNY	6	1.50 %
RAINY	->	HAZE	5	1.25 %
HAZE	->	RAINY	4	1.00 %
SNOWY	->	RAINY	3	0.75 %
HAZE	->	SNOWY	2	0.50 %
RAINY	->	SUNNY	1	0.25 %

<matplotlib.axes._subplots.AxesSubplot at 0x7fbcd7e675c0>





si può osservare come ci sia un miglioramento dell'efficienza della rete all'aumentare del numero delle epoche. Alla trentesima epoca, la rete raggiunge la massima efficienza nell'apprendere le caratteristiche delle immagini contenute nel database.

Nonostante i grafici mostrino che l'accuracy sul modello di validation del precedente era più vicino all'accuracy del training rispetto al grafico sovrastante del transfer learning, ho deciso di utilizzare quest'ultimo modello per fare la previsione, in quanto si ha accuracy più alta.

```
[ ] blindtest_datagen = ImageDataGenerator(
    rescale = 1. / 255)

blindtest_generator = blindtest_datagen.flow_from_directory(
    directory=blindtestset,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    color_mode=color_mode,
    batch_size=1,
    class_mode="categorical",
    shuffle=False
)

Found 1500 images belonging to 1 classes.

[ ] val_steps = blindtest_generator.n//blindtest_generator.batch_size+1

blind_preds = transfer_model.predict(blindtest_generator, verbose=1, steps=val_steps)
Ypred_blind = np.argmax(blind_preds, axis=1)
print(Ypred_blind)

1501/1501 [=====] - 513s 342ms/step
[3 3 3 ... 2 0 3]

[ ] import csv
Ypred_blind = map(lambda x: 'HAZE' if x==0 else ('Rainy' if x==1 else ('SNOWY' if x==2 else 'SUNNY')), Ypred_blind)
Ypred_blind = list(Ypred_blind)

with open('/content/drive/My Drive/CNN/1700170.csv', 'w') as f:
    writer = csv.writer(f)
    for val in zip(Ypred_blind):
        writer.writerow(val)
print("File successfully saved")

File successfully saved
```

Per fare la previsione, ho dovuto creare una directory per poter utilizzare il metodo `flow_from_directory` e ho salvato i risultati del test su un CSV.

CONCLUSIONI:

Il transfer learning non è sempre da preferire ad una formazione da zero, anche perché se ci sono abbastanza dati, un addestramento da zero dà risultati migliori. Nel caso di studio, il transfer learning ha dato risultati migliori ma come detto pocanzi non è sempre così.

Il limite principale della ricerca può essere il numero ridotto di esempi con cui si è allenata la rete, aumentando gli esempi. In questo modo si potrà ottimizzare il modello. Il grande svantaggio delle convolutional neural networks è la propensione al sovra adattamento, serve un'ampia mole di dati per alimentare la rete.

Possibili ricerche future per incrementare la performance del classificatore come introdurre del testo o sensori o sviluppare una rete in grado di apprendere in autonomia dai propri errori.

Bibliografia e sitografia :

1. Hijazi, Samer, Rishi Kumar, and Chris Rowen. "Using convolutional neural networks for image recognition." *Cadence Design Systems Inc.: San Jose, CA, USA* (2015): 1-12.
2. Wermter, Stefan, et al., eds. *Artificial Neural Networks and Machine Learning--ICANN 2014: 24th International Conference on Artificial Neural Networks, Hamburg, Germany, September 15-19, 2014, Proceedings*. Vol. 8681. Springer, 2014.
3. LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton. "Deep learning." *nature* 521.7553 (2015): 436-444.
4. Kim, Phil. "Convolutional neural network." *MATLAB deep learning*. Apress, Berkeley, CA, 2017. 121-147.
5. <https://keras.io/api/applications/vgg/>
6. Qassim, Hussam, Abhishek Verma, and David Feinzimer. "Compressed residual-VGG16 CNN model for big data places image recognition." *2018 IEEE 8th Annual Computing and Communication Workshop and Conference (CCWC)*. IEEE, 2018.
7. <https://neurohive.io/en/popular-networks/vgg16/>